JavaScript is disabled on your browser.



compact1, compact2, compact3

java.io

# Class FilterWriter

java.lang.Object
    java.io.Writer
        java.io.FilterWriter

**All Implemented Interfaces:**

`Closeable`, `Flushable`, `Appendable`, `AutoCloseable`

---

```
public abstract class FilterWriter
extends Writer
```

Abstract class for writing filtered character streams. The abstract class `FilterWriter` itself provides default methods that pass all requests to the contained stream. Subclasses of `FilterWriter` should override some of these methods and may also provide additional methods and fields.

**Since:**

`JDK1.1`

## *Field Summary*

**Fields**

| Modifier and Type | Field | Description |
|---|---|---|
| `protected Writer` | `out` | The underlying character-output stream. |

### Fields inherited from class java.io.Writer

lock

## *Constructor Summary*

**Constructors**

| Modifier | Constructor | Description |
|---|---|---|
| protected | `FilterWriter(Writer out)` | Create a new filtered writer. |

## *Method Summary*

**Concrete Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| void | `close()` | Closes the stream, flushing it first. |
| void | `flush()` | Flushes the stream. |
| void | `write(char[] cbuf, int off, int len)` | Writes a portion of an array of characters. |
| void | `write(int c)` | Writes a single character. |
| void | `write(String str, int off, int len)` | Writes a portion of a string. |

### Methods inherited from class java.io.Writer

append, append, append, write, write

### Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

## Field Detail

### out

```
protected Writer out
```

The underlying character-output stream.

## Constructor Detail

### FilterWriter

```
protected FilterWriter(Writer out)
```

Create a new filtered writer.

**Parameters:**

`out - a Writer object to provide the underlying stream.`

**Throws:**

`NullPointerException - if out is null`

## Method Detail

### write

```
public void write(int c)
           throws IOException
```

Writes a single character.

**Overrides:**

`write in class Writer`

**Parameters:**

`c - int specifying a character to be written`

**Throws:**

`IOException - If an I/O error occurs`

### write

```
public void write(char[] cbuf,
                  int off,
                  int len)
           throws IOException
```

Writes a portion of an array of characters.

**Specified by:**

`write` in class `Writer`

**Parameters:**

`cbuf - Buffer of characters to be written`

`off - Offset from which to start reading characters`

`len - Number of characters to be written`

**Throws:**

`IOException - If an I/O error occurs`

### write

```
public void write(String str,
                  int off,
                  int len)
           throws IOException
```

Writes a portion of a string.

**Overrides:**

`write` in class `Writer`

**Parameters:**

`str - String to be written`

`off - Offset from which to start reading characters`

`len - Number of characters to be written`

**Throws:**

`IOException - If an I/O error occurs`

### flush

```
public void flush()
           throws IOException
```

Flushes the stream.

**Specified by:**

`flush` in interface `Flushable`

**Specified by:**

`flush` in class `Writer`

**Throws:**

`IOException` - If an I/O error occurs

### close

```
public void close()
           throws IOException
```

**Description copied from class: `Writer`**

Closes the stream, flushing it first. Once the stream has been closed, further write() or flush() invocations will cause an IOException to be thrown. Closing a previously closed stream has no effect.

**Specified by:**

`close` in interface `Closeable`

**Specified by:**

`close` in interface `AutoCloseable`

**Specified by:**

`close` in class `Writer`

**Throws:**

`IOException` - If an I/O error occurs



Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.